Industrial University of Ho Chi Minh City

BÁO CÁO CUỐI KỲ · PHÂN TÍCH DỮ LIỆU LỚN · ĐỀ TÀI 8

Phân tích dữ liệu IoT và bảo trì dự đoán

Phát hiện bất thường, dự đoán RUL và cảnh báo sớm trên dữ liệu cảm biến NASA C-MAPSS

Giảng viên hướng dẫn: **TS. Võ Thị Thanh Vân**

Học viên thực hiện: **Phan Nguyễn Mai Phương · MSHV 25002711**

Ngành Khoa học máy tính · Tháng 10/2026

Bài toán: xác định thời điểm cần bảo trì cho từng động cơ

– Mỗi động cơ gửi 21 số đo cảm biến sau mỗi chu kỳ vận hành (cycle).

– Bảo trì quá sớm gây lãng phí; bảo trì quá muộn có thể dẫn đến hỏng hóc khi vận hành.

– Cần ước lượng RUL (Remaining Useful Life): số cycle còn lại trước khi hỏng.

Ví dụ về RUL

Động cơ train 1 hỏng ở cycle 192.

Tại cycle 162, RUL = 192 − 162 = 30 cycle.

Mục tiêu: xây dựng pipeline dữ liệu để phát hiện bất thường, dự đoán RUL và lập lịch bảo trì cho 100 động cơ test.

Quy trình thực hiện gồm năm bước

1. Thu thập

Tải từ NASA, kiểm tra MD5 và SHA-256

→

2. Lưu trữ

Bronze, Silver, Gold; Parquet, SQLite, HDFS

→

3. Xử lý

Pandas, PySpark, MapReduce trên YARN

→

4. Phân tích

EWMA, Health Index, mô hình RUL

→

5. Kết quả

Cảnh báo hai mức, lịch bảo trì

Dữ liệu và hạ tầng · slide 4–12

Phân tích và dự đoán · slide 13–18

Cùng một bộ dữ liệu được xử lý bằng Pandas, PySpark và MapReduce; kết quả được đối chiếu với nhau trước khi dùng cho phân tích.

Nguồn dữ liệu: bộ NASA C-MAPSS gồm bốn tập con

Tập con	Điều kiện vận hành	Dạng lỗi	Động cơ train	Động cơ test	Số bản ghi
FD001	**1**	**1 (HPC)**	**100**	**100**	**33.727**
FD002	6	1 (HPC)	260	259	87.750
FD003	1	2 (HPC, Fan)	100	100	41.316
FD004	6	2 (HPC, Fan)	249	248	102.463
Tổng			709	707	265.256

Tải bằng script từ cổng dữ liệu mở của NASA: gói 12,4 MB, 12 file; kiểm tra MD5 của gói và SHA-256 của từng file.
Phân tích chính dùng FD001 (dòng tô màu); ba tập còn lại chỉ đi qua bước ETL.

Mỗi bản ghi là một cycle của một động cơ, gồm 26 trường

engine_id	cycle	setting_1	setting_2	sensor_2	sensor_4	sensor_11	sensor_21	…
1	1	−0,0007	−0,0004	641,82	1.400,60	47,47	23,4190	…
1	2	0,0019	−0,0003	642,15	1.403,14	47,49	23,4236	…
1	3	−0,0043	0,0003	642,35	1.404,20	47,27	23,3442	…
1	4	0,0007	0,0000	642,35	1.401,87	47,13	23,3739	…

Bốn dòng đầu của train_FD001.txt; 26 trường: engine_id, cycle, setting_1–3, sensor_1–21.

train_FD001.txt

20.631 dòng
chạy đến khi hỏng

test_FD001.txt

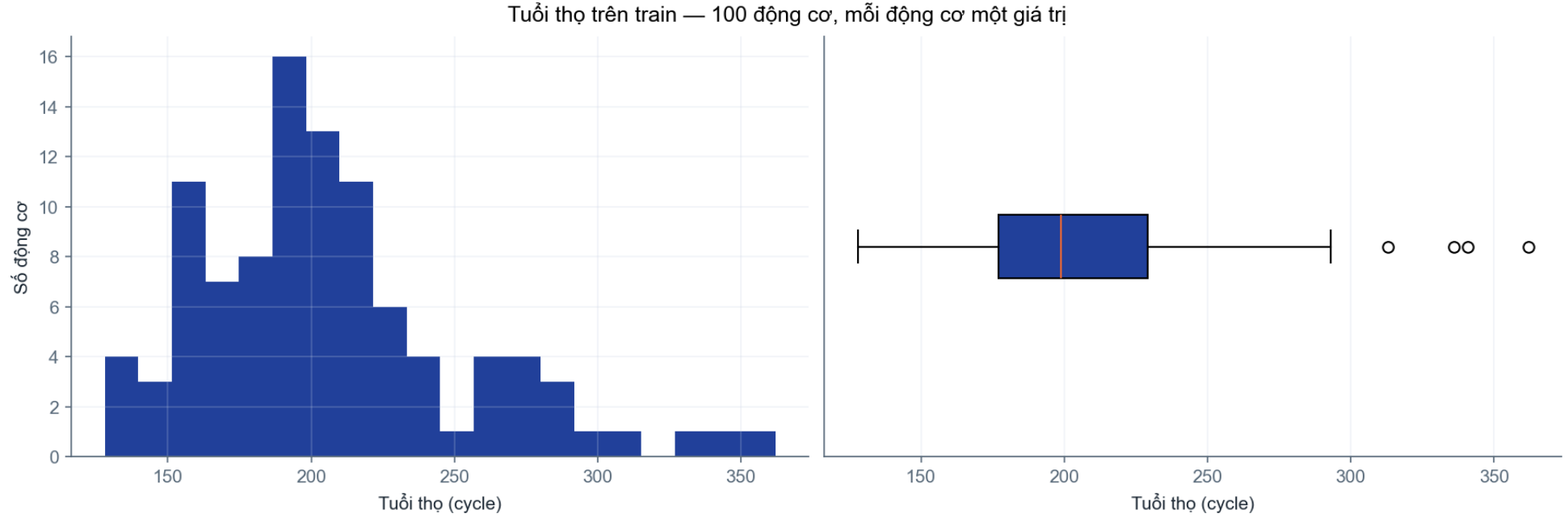
13.096 dòng
bị cắt trước khi hỏng

RUL_FD001.txt

100 dòng
RUL thật của test, chỉ dùng để chấm

Innovation - Unity - Humanity

Dữ liệu đầy đủ; tuổi thọ động cơ từ 128 đến 362 cycle



0

giá trị thiếu

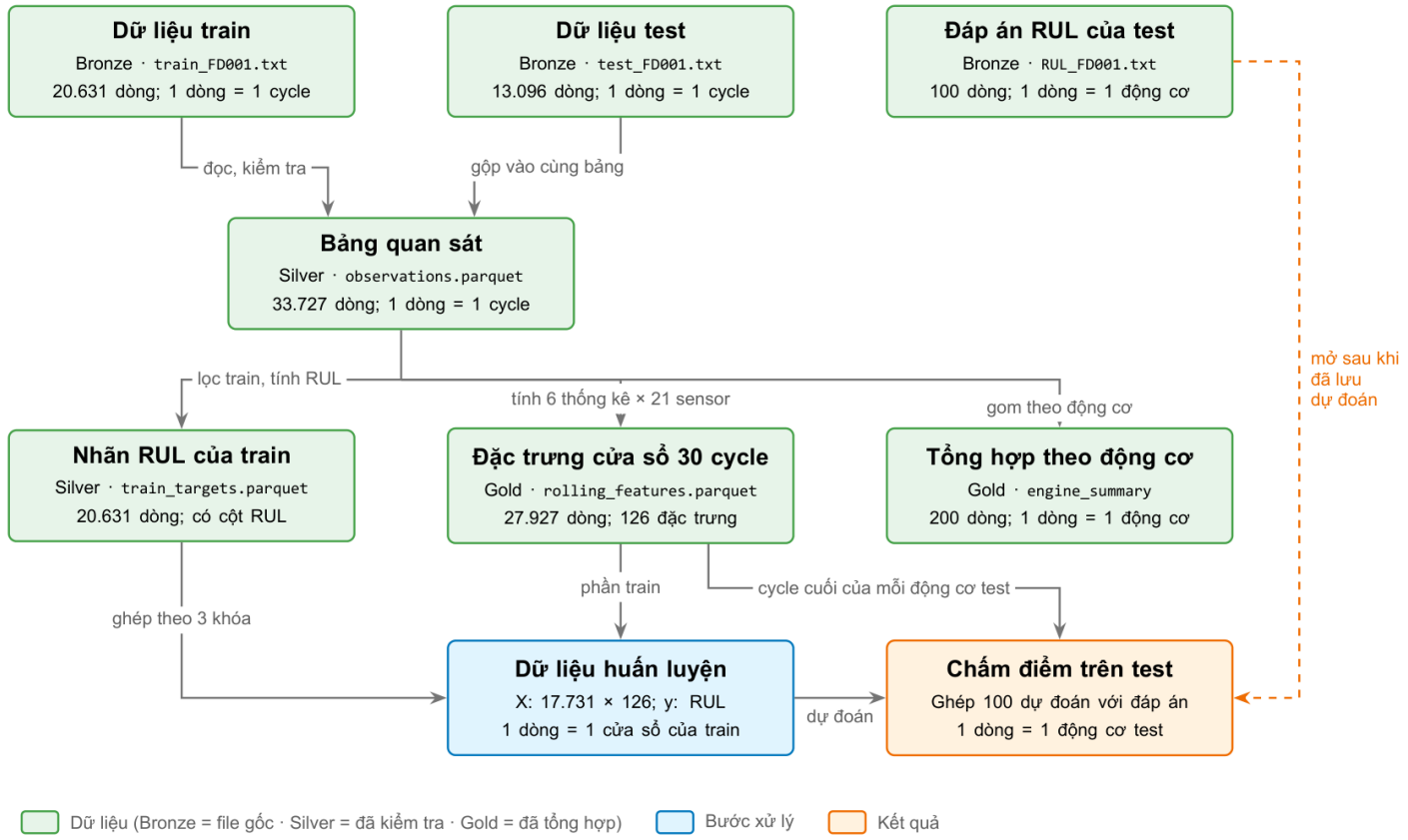
0

khóa trùng hoặc cycle bị ngắt quãng

7

cột không đổi ở cả 100 động cơ, được loại

Dữ liệu được tổ chức theo ba lớp Bronze, Silver, Gold



Bronze

File text gốc, giữ nguyên, không ghi đè

Silver

Bảng đã kiểm tra: 33.727 × 27, Parquet

Gold

Bảng tổng hợp 200 × 5 (Parquet, SQLite); đặc trưng 27.927 × 129

Innovation - Unity - Humanity

Lưu trữ và xử lý trên cụm Hadoop: HDFS, MapReduce, YARN

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	
2	0	0	2	0	<memory

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes
1	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation
Capacity Scheduler	[memory-mb (unit=Mi), vcores]	<memory:256, vCores:1>	<memory:4096, vCores:4>

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	Launch
application_1791489520331_0002	hadoop	streamjob1832214066476998265 7.jar	MAPREDUCE		root.default	0	Fri Oct 9 03:00:20 +0700 2026	Fri Oct 9 03:00:21 2026
application_1791489520331_0001	hadoop	streamjob16292415377082881816.jar	MAPREDUCE		root.default	0	Fri Oct 9 02:58:58 +0700 2026	Fri Oct 9 02:59:00 2026

Showing 1 to 2 of 2 entries

```
$ hdfs dfs -ls -R /cmapss
/cmapss/bronze/fd001/train_FD001.txt
/cmapss/silver/observations/split=train/part-00000-….snappy.parquet
$ hdfs fsck /cmapss   →   6 block, HEALTHY
```

4 task

3 map task và 1 reduce task chạy trên YARN

33.727 → 200

bản ghi đầu vào → dòng tổng hợp đầu ra

200/200

động cơ khớp với kết quả Pandas

Innovation - Unity - Humanity

PySpark cho kết quả trùng khớp với Pandas

Bảng	Số dòng	Sai khác lớn nhất	Kết quả
observations (Silver)	33.727	0	Khớp
train_targets (Silver)	20.631	0	Khớp
engine_summary (Gold)	200	0	Khớp
rolling_features (Gold)	27.927	$< 10^{-7}$	Khớp

Cùng một phép tính chạy bằng Pandas trên một máy và bằng PySpark, cho kết quả như nhau; vì vậy có thể chuyển sang Spark khi dữ liệu lớn hơn mà không đổi kết quả phân tích.

```
window = (Window
    .partitionBy("split", "engine_id")
    .orderBy("cycle")
    .rowsBetween(-29, 0))

F.avg(sensor).over(window)
F.stddev_samp(sensor).over(window)
F.min(sensor).over(window)
F.max(sensor).over(window)
F.regr_slope(sensor, "cycle").over(window)
```

Trích src/cmapss/etl/rolling_spark.py

Truy vấn dữ liệu bằng SQL trên SQLite và Spark SQL

SQL trên SQLite (lớp Gold)

```
SELECT s.engine_id, s.n_cycles, t.cycle, t.rul
FROM engine_summary AS s
JOIN train_targets AS t
  ON t.split = s.split AND t.engine_id = s.engine_id
WHERE s.split = 'train' AND s.engine_id = 1
  AND t.cycle IN (1, 100, 192);
```

engine_id	n_cycles	cycle	rul
1	192	1	191
1	192	100	92
1	192	192	0

Spark SQL trên Parquet (lớp Silver)

```
SELECT cycle, mean30 FROM (
  SELECT cycle, AVG(sensor_11) OVER w AS mean30
  FROM observations
  WHERE split = 'train' AND engine_id = 1
  WINDOW w AS (PARTITION BY split, engine_id
    ORDER BY cycle
    ROWS BETWEEN 29 PRECEDING AND CURRENT ROW))
WHERE cycle IN (30, 100);
```

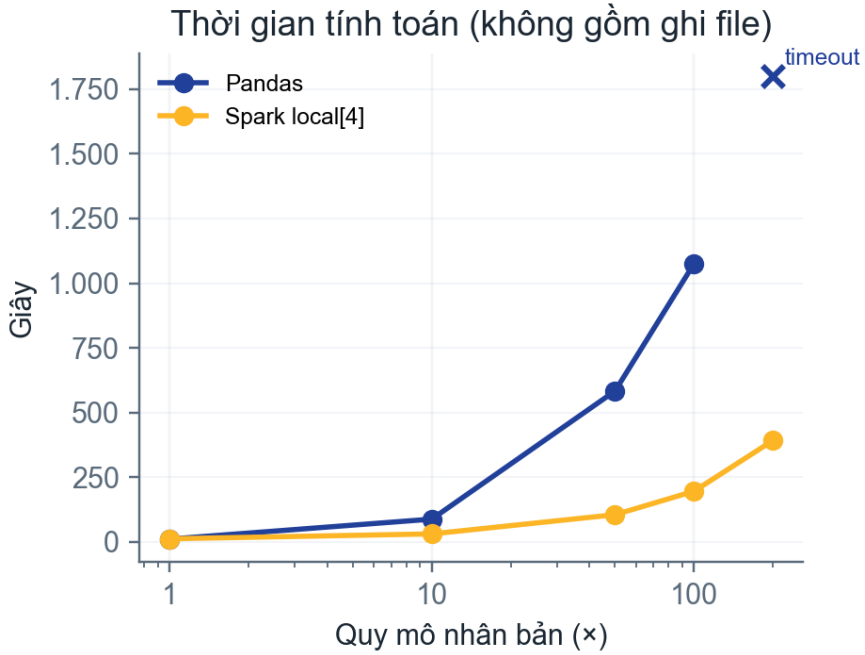
Kết quả: cycle 30 → 47,283; cycle 100 → 47,395

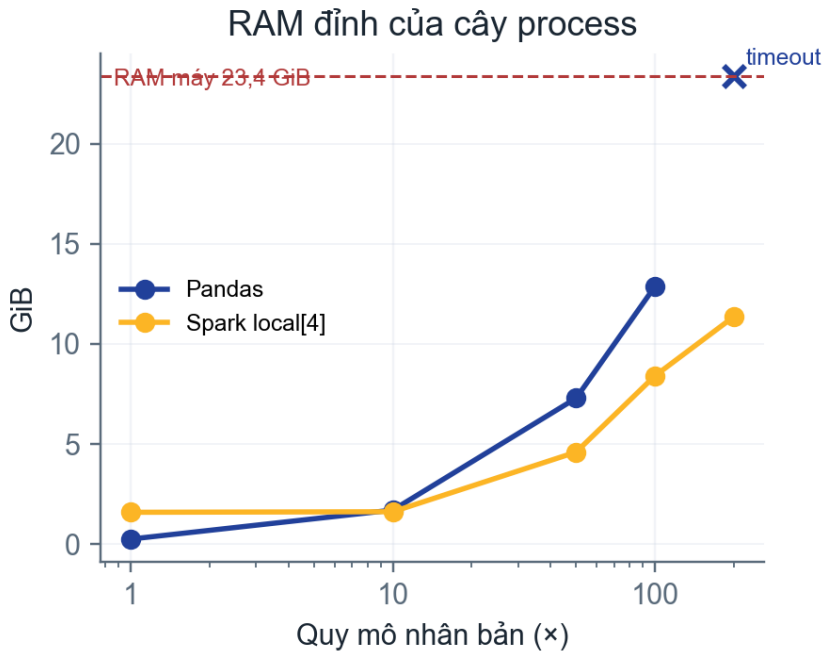
Trùng với đặc trưng sensor_11__mean do Pandas tính.

Sáu bước biến đổi từ file text thành 17.731 mẫu huấn luyện

#	Phép biến đổi	Kết quả	Kích thước
1	Đọc 2 file text, gán tên và kiểu 26 trường, thêm cột split	2 bảng có kiểu	20.631 và 13.096 dòng
2	Gộp train và test theo dòng (UNION), sắp theo khóa	observations	33.727 × 27
3	GROUP BY động cơ; RUL = cycle cuối − cycle hiện tại	train_targets	20.631 × 4
4	GROUP BY (split, engine_id): số cycle, cycle đầu, cycle cuối	engine_summary	200 × 5
5	Window 30 cycle: 6 thống kê × 21 sensor	rolling_features	27.927 × 129
6	JOIN đặc trưng train với nhãn theo đủ 3 khóa, một-một	X, y	17.731 × 126

Khi dữ liệu tăng 100 lần, Spark tính nhanh hơn Pandas 5,5 lần





5,5 lần

Spark nhanh hơn Pandas ở 100× dữ liệu

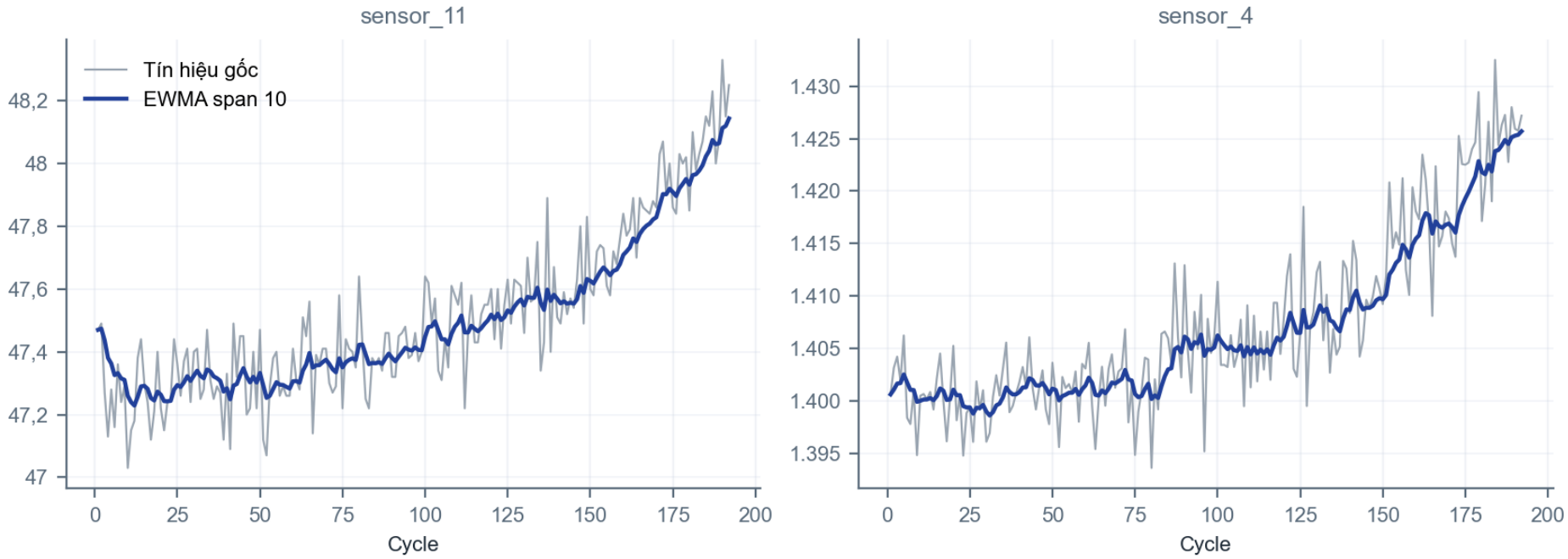
> 30 phút

Pandas ở 200×; Spark xong sau 392 giây

Như nhau

lịch bảo trì với 100 động cơ hiện tại khi dùng Pandas

Làm mượt EWMA giảm khoảng 86% dao động ngắn hạn



−86%

dao động ngắn hạn sau khi làm mượt (trung vị 14 sensor)

121 cycle

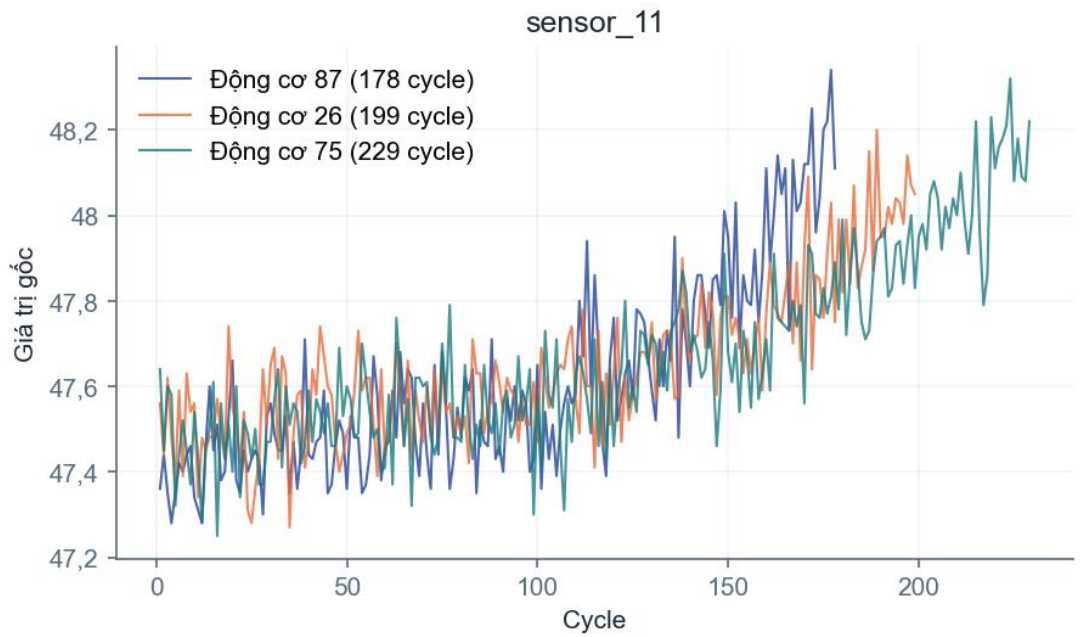
cảnh báo mức một báo trước khi hỏng, có làm mượt

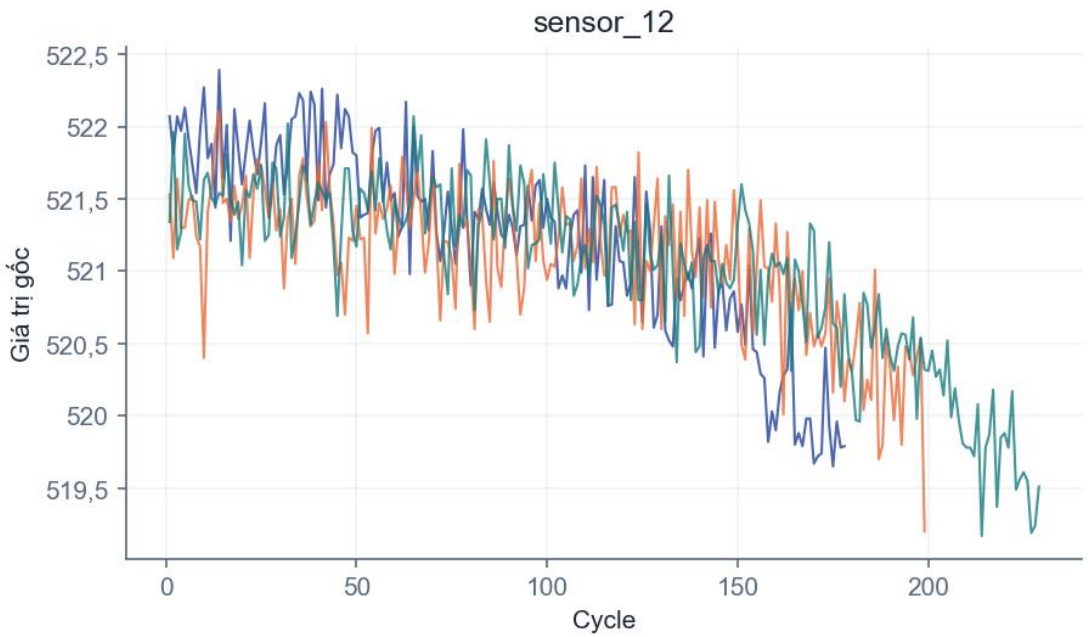
72,5 cycle

nếu không làm mượt

Innovation - Unity - Humanity

12 sensor thay đổi cùng chiều ở cả 100 động cơ khi gần hỏng





12

sensor thay đổi cùng chiều ở cả 100 động cơ

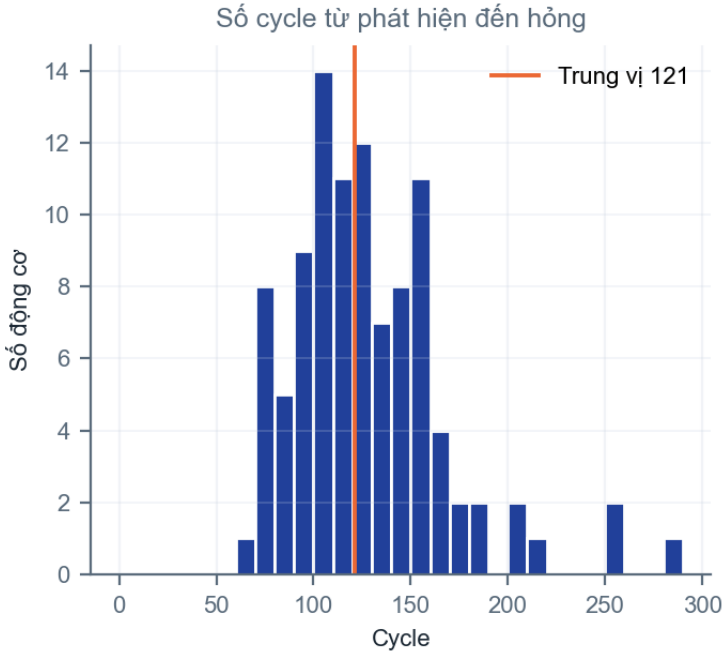
15/21

sensor thay đổi có ý nghĩa thống kê (q = 0,05)

Innovation - Unity - Humanity

Health Index phát hiện bất thường ở cả 100 động cơ train





100/100

động cơ train được phát hiện trước khi hỏng

121 cycle

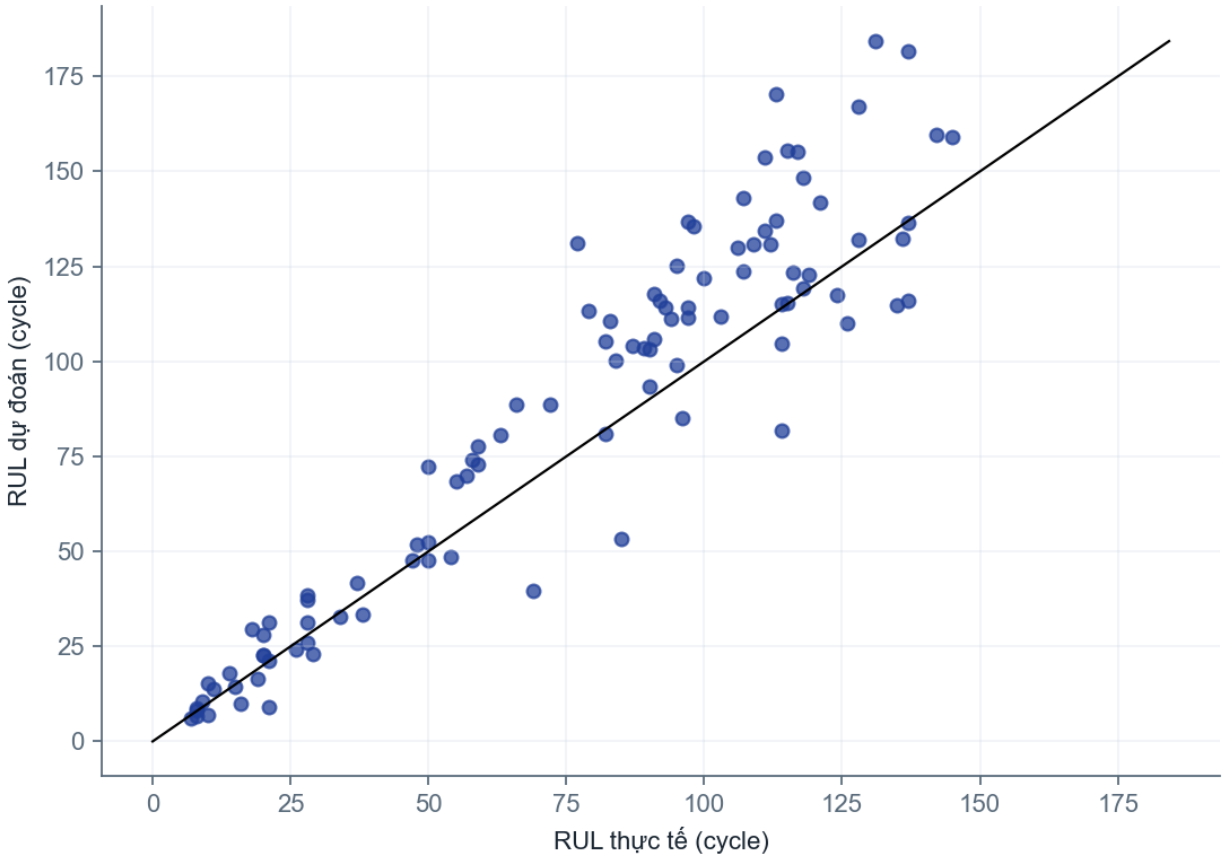
thời gian báo trước (trung vị)

cycle 84

động cơ train 1 được phát hiện; hỏng ở cycle 192

Innovation - Unity - Humanity

Mô hình HistGradientBoosting dự đoán RUL với MAE 15,35 cycle



15,35 cycle

MAE trên 100 động cơ test

4,38 cycle

MAE với 25 động cơ có RUL thật ≤ 30

20,21 cycle

MAE cross-validation 5 fold theo động cơ

Innovation - Unity - Humanity

Cảnh báo sớm: ngưỡng 40 cycle nâng tỷ lệ báo kịp thời lên 94%



49/100

động cơ train bị cảnh báo muộn ở ngưỡng 30

51% → 94%

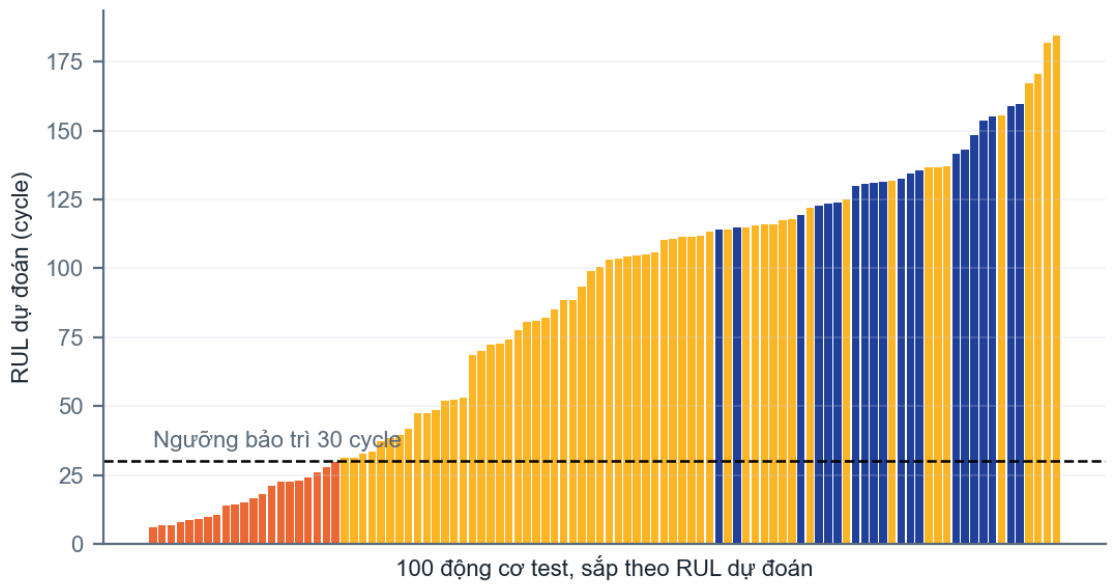
tỷ lệ cảnh báo kịp thời khi nâng ngưỡng lên 40

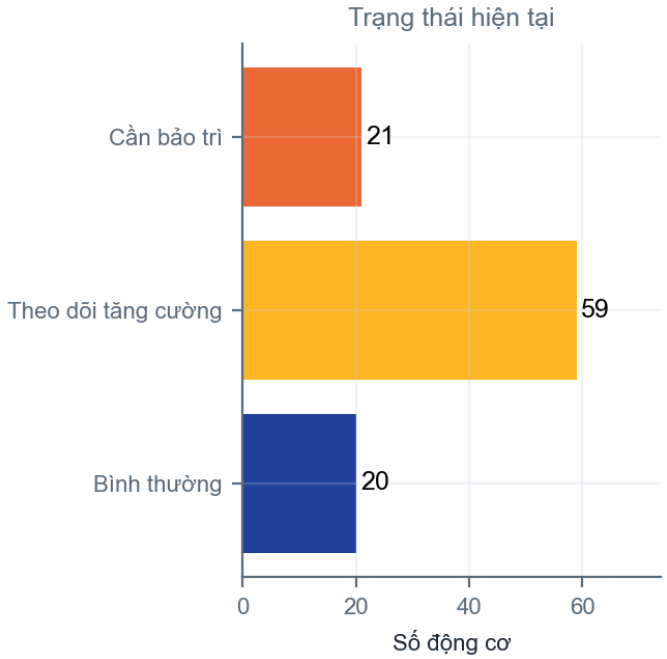
1,3% → 5,3%

tỷ lệ cycle báo quá sớm

Innovation - Unity - Humanity

Lịch bảo trì xác định đúng 21/21 động cơ cần bảo trì ngay





21/21

động cơ được xếp bảo trì ngay đều đúng

4

động cơ cần bảo trì bị sót; RUL dự đoán 31–38 cycle

25/25

bắt đủ nếu dùng ngưỡng 40 cycle

Innovation - Unity - Humanity

Kết luận, hạn chế và hướng phát triển

Kết luận

– Pipeline chạy trên Pandas, PySpark, Hadoop; kết quả khớp tuyệt đối.

– Health Index phát hiện bất thường ở 100/100 động cơ, trung vị 121 cycle trước khi hỏng.

– RUL: MAE 15,35 cycle; lịch bảo trì đúng 21/21 động cơ.

Hạn chế

– Dữ liệu mô phỏng, chưa có chi phí bảo trì.

– Mô hình thường dự đoán RUL cao hơn thực tế (74/100 động cơ test).

– Hadoop và Spark mới chạy trên một máy.

Hướng phát triển

– Chọn ngưỡng cảnh báo theo chi phí bảo trì.

– Triển khai trên cụm nhiều node.

– Nhận dữ liệu cảm biến dạng streaming.

Innovation - Unity - Humanity

Industrial University of Ho Chi Minh City



Trân trọng cảm ơn quý thầy cô đã lắng nghe

Kính mời quý thầy cô đặt câu hỏi

Giảng viên hướng dẫn: **TS. Võ Thị Thanh Vân**
Học viên: Phan Nguyễn Mai Phương · MSHV 25002711